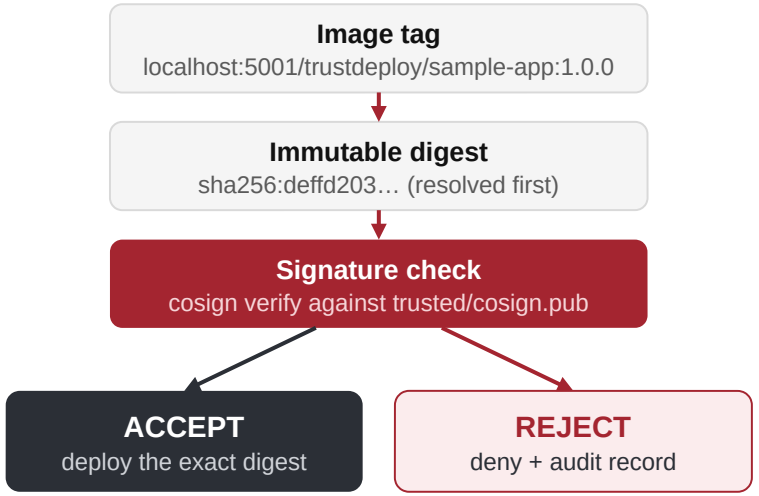
CONTAINER IMAGE SUPPLY CHAIN SECURITY USING DIGITAL SIGNATURES

TrustDeploy: Verify Before You Deploy

Integrity is the “I” in the confidentiality, integrity, availability (CIA) triad. TrustDeploy deploys a container image only when its origin is cryptographically proven, using three building blocks:

1. **Cosign signing** – approved images signed with a dedicated ECDSA P-256 key
2. **Digest-pinned verification** – tag resolved to its digest, checked against the trusted public key
3. **Fail-closed gate + audit** – ACCEPT / REJECT with reason codes, JSONL log and HTML report



Fails closed. A missing signature, an untrusted key, a tool error or an unreachable registry all mean REJECT.

GROUP MEMBERS

Under the guidance of **Prof. Sheetal Pereira**

Archit Singh
16010123057

Anish Patni
16010123045

Amrit Nigam
16010123038

Rishi Shanbhag
16010123256

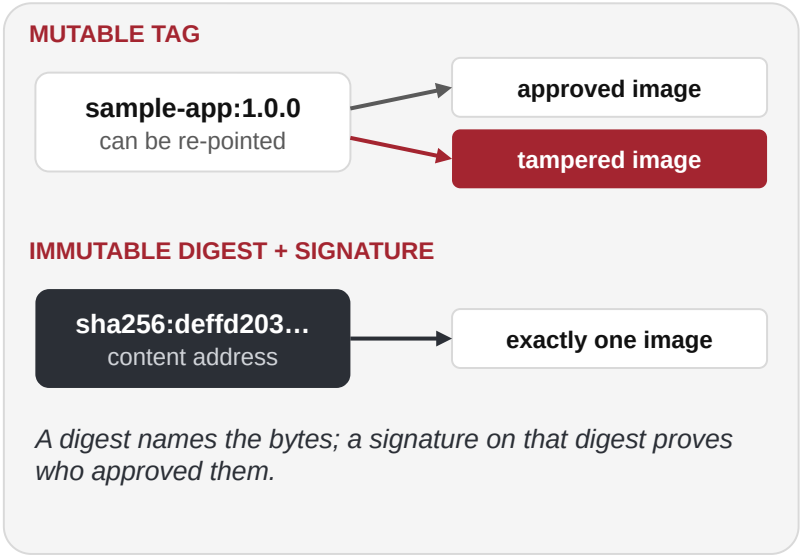
THE PROBLEM

A tag is not proof of origin

1 Compromised pipeline
A tampered build system or CI step ships an altered image under a trusted name

2 Malicious public image
Typosquatted or backdoored images pulled from a public registry

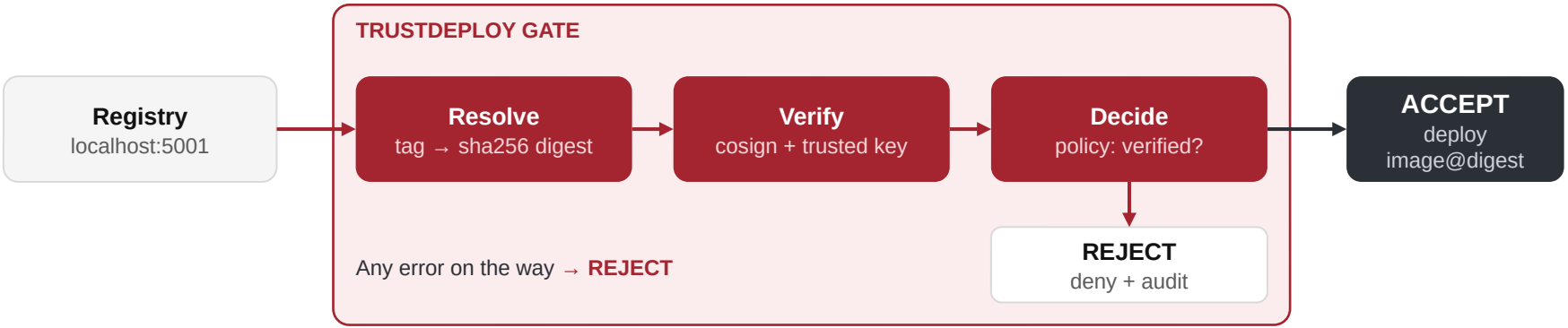
3 Image swapped after build
The registry tag is re-pointed to a different image after approval



The gap: deploying by mutable tag gives no cryptographic proof of origin, so a swapped image deploys silently.

PROPOSED SOLUTION

A verification gate before deployment



1

Fail closed

Tool error, unreachable registry or missing signature means REJECT

2

Digest-pinned

Deploys the verified sha256 digest, never the mutable tag

3

Trusted key only

Signature must verify against trusted/cosign.pub

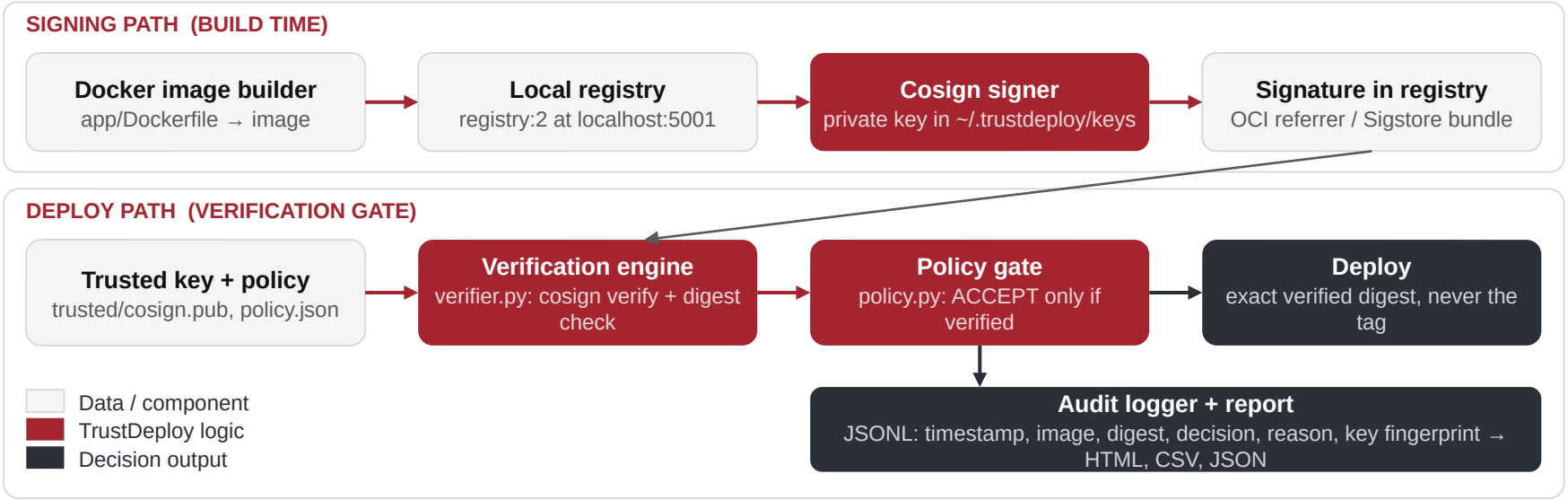
4

Fully audited

JSONL log exported as HTML dashboard, CSV and JSON

SYSTEM ARCHITECTURE

From build to verified deployment



Key isolation: the encrypted private key stays in ~/.trustdeploy/keys, outside the repo and the image; the gate trusts only the public key.

VERIFICATION ENGINE

How one ACCEPT / REJECT decision is made

1. **Registry allowed?** Image must come from a registry in policy.json
2. **Resolve tag → digest** :1.0.0 becomes sha256:… before any check
3. **cosign verify** Signature checked against trusted/cosign.pub, with a timeout
4. **Digest double-check** Signed payload digest must equal the resolved digest
5. **Policy gate** ACCEPT only if verified is True
6. **Record and deploy** Audit entry written; deploy image@sha256 digest

Fail closed by design

Cosign missing or crashes
`REJECT · VERIFICATION_TOOL_ERROR`

Registry unreachable
`REJECT · IMAGE_RESOLUTION_FAILED`

No signature attached
`REJECT · NO_SIGNATURE`

Signed by another key
`REJECT · UNTRUSTED_SIGNER`

Signature for another digest
`REJECT · SIGNATURE_IMAGE_MISMATCH`

Registry not on allow-list
`REJECT · REGISTRY_NOT_ALLOWED`

Default answer is REJECT: ACCEPT is returned only when every step succeeds and verified is True.

DECISION OUTCOMES

Seven reason codes explain every decision

REASON CODE	MEANING	DECISION
`SIGNATURE_VERIFIED`	Valid signature from the trusted key for the resolved digest	ACCEPT
`NO_SIGNATURE`	No Cosign signature is attached to the image	REJECT
`UNTRUSTED_SIGNER`	A signature exists, but not from a trusted key	REJECT
`SIGNATURE_IMAGE_MISMATCH`	Signed payload names a different digest than the image being deployed	REJECT
`IMAGE_RESOLUTION_FAILED`	Tag could not be resolved to a digest (missing image or registry down)	REJECT
`VERIFICATION_TOOL_ERROR`	Cosign failed, timed out or returned an unexpected result	REJECT
`REGISTRY_NOT_ALLOWED`	Image is from a registry outside the policy allow-list	REJECT

Traceable: every code is written to the audit log together with the image digest and key fingerprint.

TECHNOLOGY STACK

Local-first tooling with zero cloud cost

0

cloud cost

Docker, registry, Cosign and the gate all run locally

Docker 29.8 CORE
Builds the sample-app image and runs the local registry

Cosign v3.1.3 CORE
Sigstore signing and verification; signature stored as OCI referrer / bundle

Python 3 (stdlib only) CORE
verifier, policy, audit and report modules; no third-party packages

registry:2 CORE
Local OCI registry at localhost:5001 for images and signatures

kind + Policy Controller K8S
Kubernetes admission: only images signed by our key are admitted

AWS ECR OPTIONAL
Same signing and verification flow against a cloud registry

Local-first: the full sign → verify → deploy loop runs offline; cloud services are optional add-ons.

KEY MANAGEMENT AND AUDIT

Protect the signer, record every decision

Signing keys and policy

1. **Private key**
   Encrypted cosign.key in ~/.trustdeploy/keys, outside the repo and the image
2. **Public key**
   trusted/cosign.pub is the only trust anchor the gate uses
3. **Policy**
   trusted/policy.json: trusted keys, allowed registries, timeout
4. **Fingerprint**
   SHA-256 key fingerprint stored with every decision

Audit record

one JSONL line per decision

`timestamp`	`image`
`digest`	`decision`
`reason code`	`key fingerprint`
`tool evidence`	never key material

Exported reports

HTML	CSV	JSON
dashboard	spreadsheet	machine-readable

Why it matters: a reviewer can trace exactly which digest was accepted or rejected, when, and under which key.

IMPLEMENTATION EVIDENCE

Toolchain, local registry and image build

```
TrustDeploy — Phase 1: Environment setup

$ docker version --format "Docker {{.Server.Version}}"; cosign version 2>&1 | grep GitVersion; python3 --version; kind version; kubectl version --clie
  nt | head -1; echo; docker ps --filter name=td-registry --format "local registry: {{.Names}} {{.Image}} {{.Ports}}"; echo; echo "Project layout:"; f
  ind app trustdeploy trusted tests k8s scripts -type f -not -name "*.pyc" -not -name "__init__.py" | sort
Docker 29.8.1
GitVersion:    v3.1.3
Python 3.14.2
kind v0.33.0 go1.27.1 darwin/arm64
Client Version: v1.37.0

local registry: td-registry registry:2 127.0.0.1:5001->5000/tcp

Project layout:
app/app.py
app/Dockerfile
k8s/cluster-image-policy.yaml
k8s/deployment.yaml
k8s/kind-config.yaml
k8s/namespace.yaml
scripts/02_build.sh
scripts/03_keys.sh
scripts/04_sign.sh
scripts/05_verify.sh
scripts/06_prepare_untrusted.sh
scripts/07_deploy_gate.sh
scripts/08_kubernetes.sh
scripts/audit_tail.py
scripts/forge_signature.py
scripts/htmlshot.sh
scripts/retag.py
scripts/run_all.sh
scripts/termshot.py
tests/cases.sh
tests/test_gate.py
trustdeploy/audit.py
trustdeploy/cli.py
trustdeploy/policy.py
trustdeploy/report.py
trustdeploy/verifier.py
trusted/approved-image.txt
trusted/cosign.pub
trusted/policy.json
[exit code 0]  captured 2026-10-20 23:39:19 IST  host: local macOS
```

Phase 1: toolchain versions, local registry, project layout

```
TrustDeploy — Phase 2: Build image & record digest

$ ./scripts/02_build.sh
Built and pushed: localhost:5001/trustdeploy/sample-app:1.0.0
Immutable digest: sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1

REPOSITORY                              TAG              IMAGE ID       SIZE
localhost:5001/trustdeploy/sample-app   1.0.0-hijack     8c2dddd74e9a   87.9MB
localhost:5001/trustdeploy/sample-app   1.0.2-forged     15fec4ab6a77   87.9MB
localhost:5001/trustdeploy/sample-app   1.0.1-attacker   d5b8709acdb3   87.9MB
localhost:5001/trustdeploy/sample-app   1.1.0-unsigned   b9e40137b00c   87.9MB
localhost:5001/trustdeploy/sample-app   1.0.0            deffd203e38f   87.9MB

Smoke test of the image:
{"service": "trustdeploy-sample-app", "version": "1.0.0", "status": "ok"}
[exit code 0]  captured 2026-10-20 23:39:23 IST  host: local macOS
```

Phase 2: build, push and record the digest

Registry: td-registry (registry:2) on 127.0.0.1:5001

Approved image: sample-app:1.0.0, digest sha256:deffd203…

Smoke test: service returns status “ok”

IMPLEMENTATION EVIDENCE

Key generation and signing the approved image

```
TrustDeploy — Phase 3: Generate trusted signing key

$ ./scripts/03_keys.sh
Private key (encrypted, never in repo/image):
  total 24
  -rw-------@ 1 amritnigam  staff  653 20 Oct 23:30 cosign.key
  -rw-r--r--@ 1 amritnigam  staff  178 20 Oct 23:30 cosign.pub
  -rw-------@ 1 amritnigam  staff   33 20 Oct 23:30 password

Trusted public key committed to the project: trusted/cosign.pub
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEY2b6FmBx0DgxS6U62bBEmCVBe5CK
AV4VHFUgHPELCYOUwm5WMSa4Y9tspN9Mj7+8kKT+Adohn6AeA8R/AHH3BA==
-----END PUBLIC KEY-----
SHA-256 fingerprint: e40f4da2c7a6c386

Is the private key anywhere in the project or the image?
  *.key files in project tree: 0
  *.key files inside image: 0

[exit code 0]  captured 2026-10-20 23:39:24 IST  host: local macOS
```

Phase 3: encrypted key pair; public key trusted

```
TrustDeploy — Phase 3: Sign approved image with Cosign

$ ./scripts/04_sign.sh
Approved image (digest-pinned): localhost:5001/trustdeploy/sample-app@sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1

-- Sigstore bundle format (OCI referrer), used by the TrustDeploy gate
Signing artifact...
Pushing signature to: localhost:5001/trustdeploy/sample-app
-- legacy .sig format, read by the Kubernetes Policy Controller admission webhook
Signing artifact...
Pushing signature to: localhost:5001/trustdeploy/sample-app

Signature artifacts now attached to the image in the registry:
 Supply Chain Security Related artifacts for an image: localhost:5001/trustdeploy/sample-app@sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184
  d50bbe4a4c28b1
└─  Signatures for an image tag: localhost:5001/trustdeploy/sample-app:sha256-deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1.sig
   └─  sha256:a25e7b3e5194fe2c1025f65ae735a13da573a028d07b95105a0cc0c7645a98ae
└─  https://sigstore.dev/cosign/sign/v1 artifacts via OCI referrer: localhost:5001/trustdeploy/sample-app@sha256:aca95f82041925223ee91ce893126adf1b72
  c9544f5b48a096b2517f070ea2f9
   └─  sha256:a06dbb22d0d96ac2f86da7430a70bb40bc8ced1f0dcbd5ded6b5a18eeb0d8e68

[exit code 0]  captured 2026-10-20 23:39:26 IST  host: local macOS
```

Phase 3: digest-pinned signing, two signature formats

Leak check: 0 *.key files in the project tree, 0 inside the image

Trusted key fingerprint: e40f4da2c7a6c386

Signed: sample-app@sha256:deffd203… as Sigstore bundle (OCI referrer) and legacy .sig

IMPLEMENTATION EVIDENCE

TC01: a correctly signed image is accepted

```
TrustDeploy — Phase 4: Verify trusted image (TC01)

$ ./scripts/05_verify.sh
== cosign verify against trusted/cosign.pub ==

Verification for localhost:5001/trustdeploy/sample-app@sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1 --
The following checks were performed on each of these signatures:
  - The cosign claims were validated
  - Existence of the claims in the transparency log was verified offline
  - The signatures were verified against the specified public key

[{"critical":{"identity":{"docker-reference":"localhost:5001/trustdeploy/sample-app@sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4
  c28b1"},"image":{"docker-manifest-digest":"sha256:

== TrustDeploy gate, given the mutable tag :1.0.0 (resolved to digest first) ==
TrustDeploy policy gate  (policy: trustdeploy-default)
  image     : localhost:5001/trustdeploy/sample-app:1.0.0
  digest    : sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1
  decision  : ACCEPT  [SIGNATURE_VERIFIED]
  reason    : Valid signature from trusted key trusted/cosign.pub for sha256:deffd203e38f…
  audit id  : ebf0c89c-c68d-4bae-9255-50d3c4480a32  @ 2026-10-20T18:09:27+00:00

[exit code 0]  captured 2026-10-20 23:39:27 IST  host: local macOS
```

Phase 4: cosign verify, then the TrustDeploy gate on tag :1.0.0

OBSERVED RESULT

ACCEPT

`SIGNATURE_VERIFIED`

Tag resolved first: :1.0.0 → sha256:deffd203…

Key: trusted/cosign.pub

Audited: id ebf0c89c… with timestamp

Takeaway: the gate verified the digest the tag pointed to, not the tag itself, before accepting.

IMPLEMENTATION EVIDENCE

Attacks rejected: wrong key, copied signature

```
TrustDeploy — Phase 5: TC03 untrusted signing key

$ tests/cases.sh TC03
TC03 — image signed with an attacker's (untrusted) key

TrustDeploy policy gate  (policy: trustdeploy-default)
  image     : localhost:5001/trustdeploy/sample-app:1.0.1-attacker
  digest    : sha256:d5b8709acdb347ffe5db0d485a93d6cc96bedec5ae0049dba426bb70d8f31678
  decision  : REJECT  [UNTRUSTED_SIGNER]
  reason    : Signature(s) exist, but none validate against the trusted public key.
  evidence  : error during command execution: no matching attestations: failed to verify signature: could not verify envelope: accepted signatures do
  not
  audit id  : 8464a05d-d2fa-49e6-b6ab-5826ed098001  @ 2026-10-20T18:09:36+00:00
RESULT TC03: PASS (expected REJECT, got REJECT)

[exit code 0]  captured 2026-10-20 23:39:36 IST  host: local macOS
```

TC03: image signed with the attacker's key

```
TrustDeploy — Phase 5: TC04a signature copied to another image

$ tests/cases.sh TC04a
TC04a — genuine trusted signature copied onto a different image

TrustDeploy policy gate  (policy: trustdeploy-default)
  image     : localhost:5001/trustdeploy/sample-app:1.0.2-forged
  digest    : sha256:15fec4ab6a7720deff992d74b5b06dd132fecaa3847144a7a5a5562023db1fbe
  decision  : REJECT  [SIGNATURE_IMAGE_MISMATCH]
  reason    : A signature exists, but it was issued for a different image digest.
  evidence  : error during command execution: no matching attestations: failed to verify signature: provided artifact digest does not match any digest
   in
  audit id  : d6f46d81-ed61-469f-af79-6d79f98f33cb  @ 2026-10-20T18:09:36+00:00
RESULT TC04a: PASS (expected REJECT, got REJECT)

[exit code 0]  captured 2026-10-20 23:39:36 IST  host: local macOS
```

TC04a: genuine signature copied onto a different image

TC02	**Unsigned image**	REJECT · NO_SIGNATURE
TC03	**Attacker's own key**	REJECT · UNTRUSTED_SIGNER
TC04a	**Copied genuine signature**	REJECT · SIGNATURE_IMAGE_MISMATCH
TC04b	**Tampered build over :1.0.0**	REJECT · NO_SIGNATURE

IMPLEMENTATION EVIDENCE

Fail closed: errors deny deployment

```
TrustDeploy — Phase 5: TC05b registry unreachable

$ tests/cases.sh TC05b
TC05b — registry unreachable during verification of a VALID image

registry stopped

TrustDeploy policy gate  (policy: trustdeploy-default)
  image     : localhost:5001/trustdeploy/sample-app:1.0.0
  digest    : -
  decision  : REJECT  [IMAGE_RESOLUTION_FAILED]
  reason    : Could not resolve the image to a digest; failing closed.
  evidence  : ERROR: failed to do request: Head "http://localhost:5001/v2/trustdeploy/sample-app/manifests/1.0.0": dial tcp [::1]:5001: connect: conne
  ctio
  audit id  : 803b9b98-eaac-4213-8c82-c387a84008c4  @ 2026-10-20T18:09:38+00:00
DEPLOYMENT BLOCKED: localhost:5001/trustdeploy/sample-app:1.0.0 did not satisfy the trust policy. Nothing was deployed.
RESULT TC05b: PASS (expected REJECT, got REJECT)

(registry restarted)

[exit code 0]  captured 2026-10-20 23:39:40 IST  host: local macOS
```

TC05b: registry unreachable during verification of a valid image

```
TrustDeploy — Phase 5: Unit tests (fail-closed logic)

$ python3 -m unittest tests.test_gate -v 2>&1
test_decide_requires_explicit_true (tests.test_gate.GateTests.test_decide_requires_explicit_true) ... ok
test_failure_reasons_classified (tests.test_gate.GateTests.test_failure_reasons_classified) ... ok
test_missing_tool_fails_closed (tests.test_gate.GateTests.test_missing_tool_fails_closed) ... ok
test_payload_for_other_digest_rejected_even_if_cosign_passes (tests.test_gate.GateTests.test_payload_for_other_digest_rejected_even_if_cosign_passes)
  ... ok
test_registry_not_allowed (tests.test_gate.GateTests.test_registry_not_allowed) ... ok
test_timeout_fails_closed (tests.test_gate.GateTests.test_timeout_fails_closed) ... ok
test_unexpected_exception_fails_closed (tests.test_gate.GateTests.test_unexpected_exception_fails_closed) ... ok
test_valid_signature_accepted (tests.test_gate.GateTests.test_valid_signature_accepted) ... ok

----------------------------------------------------------------------
Ran 8 tests in 0.002s

OK

[exit code 0]  captured 2026-10-20 23:39:42 IST  host: local macOS
```

Unit tests: fail-closed decision logic (8 tests, OK)

TC05a — **Cosign missing** — `VERIFICATION_TOOL_ERROR`

TC05b — **Registry down** — `IMAGE_RESOLUTION_FAILED`

TC05c — **nginx from docker.io** — `REGISTRY_NOT_ALLOWED`

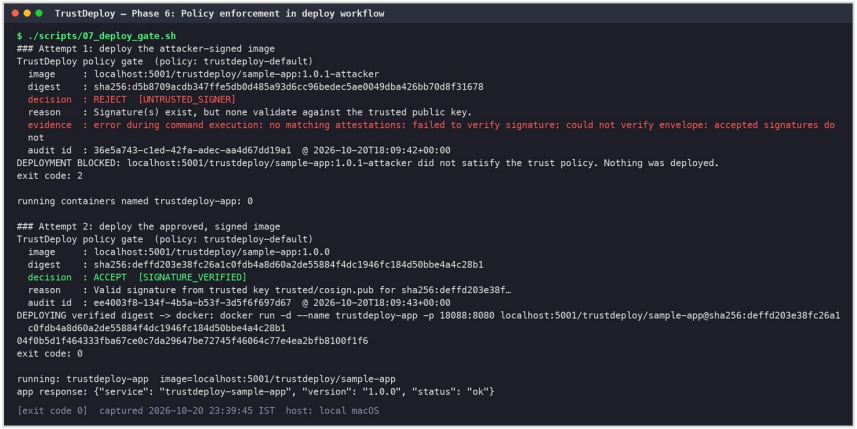
8/8

unit tests pass

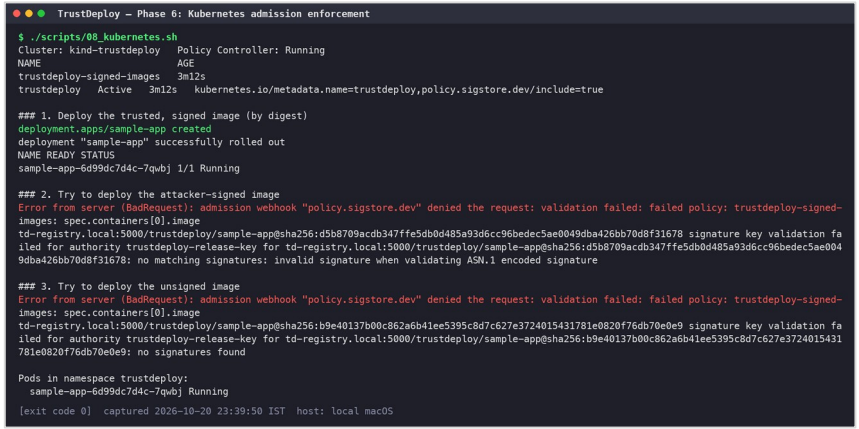
timeouts, missing tool and exceptions all fail closed

IMPLEMENTATION EVIDENCE

Enforced at deploy time: Docker and Kubernetes



Docker: attacker image blocked; verified digest deployed



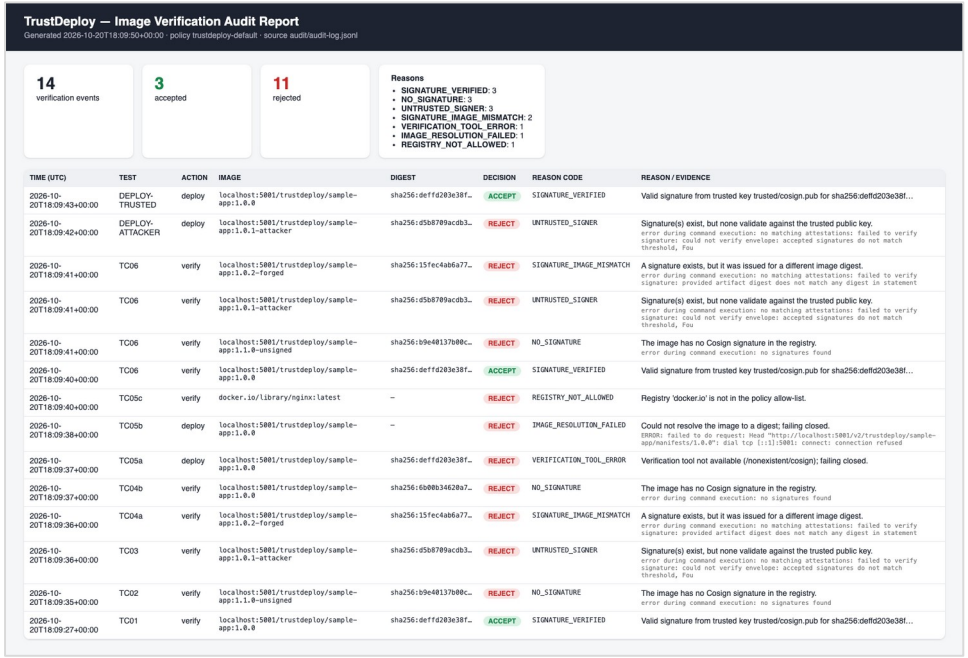
Kubernetes: Policy Controller admission on kind

Docker gate: attacker-signed image blocked (exit code 2, 0 containers); approved image run as image@sha256 digest.

Admission: signed image admitted (pod Running); attacker-signed and unsigned images denied by the webhook.

IMPLEMENTATION EVIDENCE

TC07: audit report and HTML dashboard



TrustDeploy — Image Verification Audit Report

Generated 2026-10-20T18:09:50+00:00 · policy trustdeploy-default · source audit/audit-log.jsonl

14	3	11
verification events	accepted	rejected

Reasons

- SIGNATURE_VERIFIED: 3
- NO_SIGNATURE: 3
- UNTRUSTED_SIGNER: 3
- SIGNATURE_IMAGE_MISMATCH: 2
- VERIFICATION_TOOL_ERROR: 1
- IMAGE_RESOLUTION_FAILED: 1
- REGISTRY_NOT_ALLOWED: 1

TIME (UTC)	TEST	ACTION	IMAGE	DIGEST	DECISION	REASON CODE	REASON / EVIDENCE
2026-10-20T18:09:43+00:00	DEPLOY-TRUSTED	deploy	localhost:5001/trustdeploy/sample-app:1.0.0	sha256:deffd203e38f…	ACCEPT	SIGNATURE_VERIFIED	Valid signature from trusted key trusted/cosign.pub for sha256:deffd203e38f…
2026-10-20T18:09:42+00:00	DEPLOY-ATTACKER	deploy	localhost:5001/trustdeploy/sample-app:1.0.1-attacker	sha256:d5b8709acdb3…	REJECT	UNTRUSTED_SIGNER	Signature(s) exist, but none validate against the trusted public key. error during command execution: no matching attestations: failed to verify signature: could not verify envelope: accepted signatures do not match threshold, Fou
2026-10-20T18:09:41+00:00	TC06	verify	localhost:5001/trustdeploy/sample-app:1.0.2-forged	sha256:15fec4ab6a77…	REJECT	SIGNATURE_IMAGE_MISMATCH	A signature exists, but it was issued for a different image digest. error during command execution: no matching attestations: failed to verify signature: provided artifact digest does not match any digest in statement
2026-10-20T18:09:41+00:00	TC06	verify	localhost:5001/trustdeploy/sample-app:1.0.1-attacker	sha256:d5b8709acdb3…	REJECT	UNTRUSTED_SIGNER	Signature(s) exist, but none validate against the trusted public key. error during command execution: no matching attestations: failed to verify signature: could not verify envelope: accepted signatures do not match threshold, Fou
2026-10-20T18:09:41+00:00	TC06	verify	localhost:5001/trustdeploy/sample-app:1.1.0-unsigned	sha256:b9e40137b00c…	REJECT	NO_SIGNATURE	The image has no Cosign signature in the registry. error during command execution: no signatures found
2026-10-20T18:09:40+00:00	TC06	verify	localhost:5001/trustdeploy/sample-app:1.0.0	sha256:deffd203e38f…	ACCEPT	SIGNATURE_VERIFIED	Valid signature from trusted key trusted/cosign.pub for sha256:deffd203e38f…
2026-10-20T18:09:40+00:00	TC05c	verify	docker.io/library/nginx:latest	–	REJECT	REGISTRY_NOT_ALLOWED	Registry 'docker.io' is not in the policy allow-list.
2026-10-20T18:09:38+00:00	TC05b	deploy	localhost:5001/trustdeploy/sample-app:1.0.0	–	REJECT	IMAGE_RESOLUTION_FAILED	Could not resolve the image to a digest; failing closed. ERROR: failed to do request: Head "http://localhost:5001/v2/trustdeploy/sample-app/manifests/1.0.0": dial tcp [::1]:5001: connect: connection refused
2026-10-20T18:09:37+00:00	TC05a	deploy	localhost:5001/trustdeploy/sample-app:1.0.0	sha256:deffd203e38f…	REJECT	VERIFICATION_TOOL_ERROR	Verification tool not available (/nonexistent/cosign); failing closed.
2026-10-20T18:09:37+00:00	TC04b	verify	localhost:5001/trustdeploy/sample-app:1.0.0	sha256:6b00b34620a7…	REJECT	NO_SIGNATURE	The image has no Cosign signature in the registry. error during command execution: no signatures found
2026-10-20T18:09:36+00:00	TC04a	verify	localhost:5001/trustdeploy/sample-app:1.0.2-forged	sha256:15fec4ab6a77…	REJECT	SIGNATURE_IMAGE_MISMATCH	A signature exists, but it was issued for a different image digest. error during command execution: no matching attestations: failed to verify signature: provided artifact digest does not match any digest in statement
2026-10-20T18:09:36+00:00	TC03	verify	localhost:5001/trustdeploy/sample-app:1.0.1-attacker	sha256:d5b8709acdb3…	REJECT	UNTRUSTED_SIGNER	Signature(s) exist, but none validate against the trusted public key. error during command execution: no matching attestations: failed to verify signature: could not verify envelope: accepted signatures do not match threshold, Fou
2026-10-20T18:09:35+00:00	TC02	verify	localhost:5001/trustdeploy/sample-app:1.1.0-unsigned	sha256:b9e40137b00c…	REJECT	NO_SIGNATURE	The image has no Cosign signature in the registry. error during command execution: no signatures found
2026-10-20T18:09:27+00:00	TC01	verify	localhost:5001/trustdeploy/sample-app:1.0.0	sha256:deffd203e38f…	ACCEPT	SIGNATURE_VERIFIED	Valid signature from trusted key trusted/cosign.pub for sha256:deffd203e38f…

14 verification events

3 accepted

11 rejected

Exports: report.html, report.csv, report.json

Per record: digest, decision, reason code, key fingerprint, tool evidence

audit/report.html, generated from audit/audit-log.jsonl

TESTING

Test results: every attack was rejected

ID	SCENARIO	EXPECTED	OBSERVED
TC01	Correctly signed image	**Accepted**	ACCEPT · SIGNATURE_VERIFIED
TC02	Unsigned image	**Rejected**	REJECT · NO_SIGNATURE
TC03	Signed with the attacker's key	**Rejected**	REJECT · UNTRUSTED_SIGNER
TC04a	Genuine signature copied onto another image	**Rejected**	REJECT · SIGNATURE_IMAGE_MISMATCH
TC04b	Trusted tag re-pointed to a tampered build	**Rejected**	REJECT · NO_SIGNATURE
TC05a	Verification tool (Cosign) missing	**Deployment denied**	REJECT · VERIFICATION_TOOL_ERROR
TC05b	Registry down during verification	**Deployment denied**	REJECT · IMAGE_RESOLUTION_FAILED
TC05c	Image from a registry outside the policy	**Rejected**	REJECT · REGISTRY_NOT_ALLOWED
TC06	Multiple images checked in one run	**Each recorded separately**	PASS · 4 separate audit entries
TC07	Audit report export	**Report generated**	PASS · HTML, CSV, JSON written

8/8
unit tests pass

K8s
admission: signed admitted, attacker & unsigned denied

LIMITATIONS

What a signature does not prove

Not a vulnerability scan

A signature proves origin and integrity under the policy, not that the image is free of vulnerabilities.

Depends on key security

If the private signing key is stolen, an attacker can produce signatures the gate will trust.

Transparency log disabled offline

The offline prototype skips the public Rekor log, so signing events are not publicly recorded.

No malware scanning

The contents of a correctly signed image are not inspected for malicious code.

Scope: TrustDeploy answers “is this the image we approved?” — not “is this image safe?”

FUTURE WORK

Hardening toward a production pipeline

CI/CD pipeline

1. GitHub Actions signing
2. Vulnerability scan (e.g. Trivy) before signing
3. SBOM generation

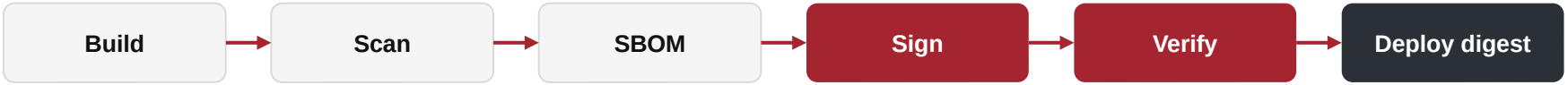
Provenance

4. SLSA provenance attestations
5. Keyless signing with OIDC (Sigstore)

Keys and platform

6. KMS-backed keys and rotation
7. Amazon ECR / EKS deployment

TARGET PIPELINE



CONCLUSION

Only proven images reach deployment

1

rule decides

ACCEPT only if verified is True; everything else is REJECT

What TrustDeploy delivers

- ✓ Cosign signing with a dedicated ECDSA P-256 key
- ✓ Tag resolved to digest before every check
- ✓ Signature and signed-digest double-check
- ✓ Fail-closed gate with seven reason codes
- ✓ JSONL audit with HTML, CSV and JSON reports
- ✓ Same rule enforced at Kubernetes admission

Next steps

CI signing
GitHub Actions

Scan + SBOM
Trivy before signing

Provenance
SLSA attestations

Keys
keyless OIDC or KMS with rotation

Sign
approved image, by digest → **Verify**
trusted key + digest check → **Deploy**
exact verified digest